A Reproducibility Benchmark of the FlashAttention Family Against Classical and Approximate Attention

Bahaa Al-Juboori
British Columbia Institute of Technology
Burnaby, BC, Canada

Shaaz E Jahan
British Columbia Institute of Technology
Burnaby, BC, Canada

Jason Fakidis
British Columbia Institute of Technology
Burnaby, BC, Canada

Aneet Kaur
British Columbia Institute of Technology
Burnaby, BC, Canada

Abstract

Transformer self-attention costs time and memory that grow quadratically in sequence length, and on modern GPUs this cost is dominated by data movement rather than arithmetic. The FlashAttention family addresses this with IO-aware exact attention: it reorganizes the computation to respect the GPU memory hierarchy, achieving large speedups and linear memory with no approximation. We propose to benchmark this family, from the original FlashAttention through its hardware-specialized successors, against traditional exact and approximate baselines. Using single-GPU micro-benchmarks that sweep sequence length, masking, and pass type, we measure runtime, throughput, peak memory, maximum context before out-of-memory, and numerical accuracy. Our goal is to test whether the claimed advantages hold on accessible hardware, to isolate how much of the advantage comes specifically from IO-awareness, and to document the practical limits of adoption, including the alpha-stage status of the newest kernel and the degree to which production libraries have already absorbed its techniques.

Keywords

attention, Transformers, GPU kernels, IO-awareness, FlashAttention

1 Problem Definition & Motivation

1.1 Problem

Self-attention is the mechanism that lets a Transformer [15] relate every position in a sequence to every other. For a sequence of length N, each position is represented by a query, key, and value vector of dimension d, stacked into matrices $Q, K, V \in \mathbb{R}^{N \times d}$. Attention computes a score between every query and every key, normalizes these scores with a row-wise softmax, and uses them to take a weighted average of the values: $\mathrm{softmax}(QK^\top/\sqrt{d})\,V$. The intermediate score matrix $QK^\top$ therefore has one entry for every pair of positions.

That pairwise structure is the source of the cost. The score matrix is $N \times N$, so forming it, applying softmax, and multiplying by V take time and memory that grow quadratically in N. On a modern GPU the binding constraint is not the arithmetic but the *memory traffic*: the processor's compute throughput far exceeds its memory bandwidth, and the memory hierarchy is steeply tiered, with a small, fast on-chip SRAM sitting above a large, slow high-bandwidth memory (HBM). A naive implementation writes the full $N \times N$ matrix to HBM and reads it back, so attention becomes *memory-bound*: its speed is governed by moving the quadratic intermediate to and from HBM, not by the floating-point operations.

The consequence is that long context is expensive precisely where it is most useful. As N grows, the quadratic memory and traffic quickly dominate runtime and exhaust GPU memory, which has historically capped the sequence lengths Transformers can handle. The problem we study is whether this bottleneck can be removed *without* changing the function computed, by reorganizing the computation to respect the memory hierarchy rather than approximating attention, and, concretely, whether the FlashAttention family delivers that in practice and under what conditions.

1.2 Motivation

Long context has become central to modern machine learning: large language models, long-document understanding, and high-resolution vision all depend on attending over many thousands of positions, and the quadratic cost of attention is the binding constraint on how far they can scale. Making exact attention faster and more memory-efficient therefore pays off broadly, improving latency, throughput, and the maximum context that fits on a given GPU. Crucially, the IO-aware approach achieves this *without* approximating attention, so it avoids the accuracy penalty that sub-quadratic methods incur; where it applies, it is close to a free improvement, which is why it has largely displaced earlier efficiency techniques in practice.

This makes the FlashAttention family worth studying empirically rather than taking its advantages for granted. Its performance claims are reported by the authors on specific, often high-end, hardware, while the production libraries (cuDNN and PyTorch SDPA) have progressively absorbed the same techniques, so how much advantage remains in practice is an open question. The newest version, FlashAttention-4, is both very recent and an alpha-stage release, which raises real questions of software maturity, numerical precision, and reproducibility. An independent benchmark on accessible hardware, spanning the lineage from the original algorithm to its latest variant, is therefore a useful contribution: it tests

whether the claimed benefits hold, isolates where they come from, and documents the practical cost of adopting them.

2 Literature Review

We review the lines of research that led to our modern algorithm, following the intellectual lineage that FlashAttention [6] itself traces. Each paragraph treats one line of work; a theoretical comparison and the supporting mathematics are given in the supplementary appendices.

Origins of attention and the quadratic problem. Attention originated as a memory-programming idea: Schmidhuber's fast-weight controllers, in which a slow network learns to generate the weights of a fast network through outer products [13], were later reframed explicitly as "attention to the recent past" [2]. The modern, content-based form was introduced for neural machine translation [3] and then made the center of an architecture by the Transformer [15]. The Transformer's self-attention, however, has time and memory cost quadratic in sequence length N, which is the bottleneck all later work addresses.

IO-aware runtime optimization. A separate, older line from systems and computer architecture studies how to optimize reads and writes across a memory hierarchy. The input/output complexity model [1] formalizes counting accesses to slow versus fast memory, and the Roofline model [17] frames performance in terms of arithmetic intensity against memory bandwidth. Applied to Transformers, this perspective shows that data movement, not arithmetic, is the dominant cost [8]. This line supplies the key principle that our modern algorithm brings to attention.

Approximate (sub-quadratic) attention. The first response to the quadratic cost was to approximate attention. Sparse and hashing-based methods such as Reformer reduce cost by attending to a subset [10], while low-rank and kernel methods achieve linear complexity: Performer uses random features [4], Linformer projects to a low-rank subspace [16], and linear attention reformulates the computation as a recurrence [9]. These methods reduce FLOPs but often fail to deliver wall-clock speedup, because theoretical efficiency does not automatically translate to hardware efficiency, a mismatch known as the hardware lottery [7]. This gap is precisely what motivates an IO-aware, exact approach.

Exact, memory-reduced attention. A second response keeps attention exact and attacks memory instead. The online softmax normalizer computes the softmax incrementally over blocks without materializing a full row [11], the primitive that makes tiled exact attention possible. Rabe and Staats then showed self-attention needs only sub-quadratic *memory* while remaining exact, though its time stays $O(N^2)$ [12]. These exact-but-not-IO-aware methods are the natural comparison points for our modern algorithm.

Synthesis: the FlashAttention lineage. FlashAttention [6] unifies IO-awareness with exact attention, using tiling and online softmax to keep the $N \times N$ matrix out of high-bandwidth memory. The result is exact (not an approximation), faster in wall-clock time, and linear in memory. Its stated limitation, that each kernel must be hand-written in CUDA, drove the subsequent hardware co-design line: FlashAttention-2 [5], FlashAttention-3 [14], and our modern algorithm FlashAttention-4 [18], which this project benchmarks.

3 Methods Under Study

This section identifies the modern algorithm we study and the baselines we compare it against: Section 3.1 selects the FlashAttention family and explains why it is an advancement, and Section 3.2 lists the traditional methods used for comparison.

3.1 Modern Algorithm Selection

Choice of modern algorithm. We select the FlashAttention family of IO-aware exact-attention algorithms as our modern algorithm, centering our study on the original FlashAttention (FA1) [6], introduced in 2022. FA1 established the core principle the whole family shares: computing exact attention while minimizing data movement between GPU high-bandwidth memory and on-chip SRAM, using tiling together with an online-softmax formulation. We treat FA1 as the primary object of study because it defines this idea, and we additionally examine its modern variants: FlashAttention-2 (2023) [5], FlashAttention-3 (2024) [14], and FlashAttention-4 (2026) [18], which keep FA1's algorithmic core unchanged while re-engineering the kernel for successive GPU generations.

Why it is an advancement. FA1's central advance is a change of perspective: because attention is memory-bound rather than compute-bound on GPUs, the cost that matters is data movement between high-bandwidth memory (HBM) and on-chip SRAM, not arithmetic. By tiling the computation and using an online-softmax formulation, FA1 avoids ever writing the $N \times N$ score matrix to HBM, which reduces memory from quadratic to linear in sequence length and cuts HBM traffic accordingly [6]. Crucially this is achieved with *no approximation*: FA1 computes exactly the same function as standard attention, so it delivers wall-clock speedups (up to 7.6× on GPT-2 in the original paper) without the accuracy trade-off that the approximate methods in Section 2 incur, and it enables context lengths that were previously infeasible. Each later version keeps this exact IO-aware core and targets the next hardware bottleneck: FlashAttention-2 [5] improves work partitioning across thread blocks and warps and reduces non-matmul operations, roughly doubling throughput on A100; FlashAttention-3 [14] exploits Hopper asynchrony (warp specialization and overlapped matmul/softmax) and FP8 low precision; and FlashAttention-4 [18] co-designs the algorithm and kernel for Blackwell, overlapping matrix multiplication with the softmax and memory work that now dominate.

Application. We apply the FlashAttention family to long-sequence Transformer attention, studied as a controlled single-GPU micro-benchmark rather than full model training: we isolate one attention layer and sweep sequence length, masking, and pass type, measuring runtime, throughput, peak memory, and the maximum context length before out-of-memory. This application lets us test, on accessible GPU hardware, whether FA1's IO-aware principle and its hardware-specialized successors deliver their claimed benefits in practice.

3.2 Baseline Selection

Baselines. We compare FlashAttention against four traditional baselines that predate it and span the two established families of attention computation, exact and approximate, exceeding the three classical comparisons required for the Excellent band. These are the methods against which FlashAttention's original contribution is properly measured; the later FlashAttention versions and the production cuDNN/SDPA kernel are the same algorithmic family and are therefore used as references in the lineage study rather than as traditional baselines.

Naive scaled dot-product attention. The standard implementation [15] materializes the full $N \times N$ score matrix in GPU memory, giving $O(N^2)$ memory and memory traffic. It is the canonical "before FlashAttention" reference and the method whose quadratic cost motivated the entire line of work.

Rabe-Staats memory-efficient attention. This method [12] computes *exact* attention with sub-quadratic memory via lazy-softmax chunking, but its time stays $O(N^2)$ and it is not IO-aware. Including it lets us isolate how much of FlashAttention's advantage comes specifically from IO-aware tiling rather than chunking alone.

Approximate attention (Performer, Linformer). These represent the sub-quadratic family that trades exactness for scalability: Performer approximates the softmax kernel with random features [4], and Linformer projects keys and values to a low-rank subspace [16]. Both achieve near-linear cost but introduce an accuracy gap relative to exact attention, the trade-off FlashAttention avoids, and the basis of our exact-versus-approximate comparison.

PyTorch SDPA / cuDNN (production reference). We additionally report the fused SDPA/cuDNN backend as a strong production reference. Because modern SDPA dispatches to FlashAttention-class kernels, it is not a traditional baseline; we include it to show how a well-engineered library compares on each GPU, and to avoid overstating gains against a naive strawman.

4 Plan & Methodology

This section states the questions the study will answer and the experiments designed to answer them: Section 4.1 sets out the research questions, and Section 4.2 describes the benchmark design, swept parameters, metrics, and reproducibility measures.

4.1 Research Questions

Our research questions address the advantages claimed, the conditions under which we expect them, and the potential limitations; the experiments that will test them are described in the Experimental Plan below.

Claimed advantages. Does FlashAttention's IO-aware exact attention deliver its claimed advantages over traditional baselines, namely lower runtime, higher throughput, and $O(N)$ rather than $O(N^2)$ memory (RQ1)? We further ask how much of this advantage comes specifically from IO-aware tiling rather than chunking alone, isolated by comparison with the exact but non-IO-aware Rabe-Staats method [12] (RQ2); and whether FlashAttention matches the efficiency of approximate methods such as Performer [4] and Linformer [16] while incurring no approximation error (RQ3).

Conditions for better performance. Under what conditions is the advantage largest? We expect it to grow with sequence length, where the quadratic cost dominates, and to differ between causal and non-causal masking, since causal masking lets the kernel skip roughly half of the score blocks (RQ5). We also ask whether the advantage holds across GPU generations: does each FlashAttention version reproduce its claimed improvement over its predecessor, with FlashAttention-2 over FlashAttention-1 [5] on Ampere, FlashAttention-3 [14] on Hopper, and FlashAttention-4 [18] on Blackwell (RQ4)?

Potential limitations. What are the practical limits of adopting FlashAttention (RQ6)? These include hardware dependence (each version targets a specific GPU generation), software maturity (the FlashAttention-4 kernel is an alpha-stage release), numerical precision across BF16/FP8/FP4, and reproducibility. We further ask whether the advantage is durable: because the production cuDNN/SDPA library has progressively absorbed FlashAttention's techniques, with FlashAttention-4 reporting only a modest margin over recent cuDNN, how much benefit remains for the research kernel on current hardware (RQ7)?

4.2 Experimental Plan

We evaluate FlashAttention through controlled single-GPU micro-benchmarks of one attention layer. The plan comprises a primary study against traditional baselines (RQ1 to RQ3), a per-generation lineage study (RQ4), and shared limitation analyses (RQ6 and RQ7).

Methods. The methods under study are FlashAttention-1/2, FlashAttention-3 (Hopper build), and FlashAttention-4. Traditional baselines are naive attention [15], Rabe-Staats memory-efficient attention [12], and the approximate methods Performer [4] and Linformer [16]; PyTorch SDPA/cuDNN serves as a production reference.

Swept parameters. We sweep three parameters directly: sequence length $N \in \{512, 1K, 2K, 4K, 8K, 16K, 32K, 64K\}$ (extended until out-of-memory); masking (causal vs. non-causal), since causal masking skips roughly half the score blocks; and pass type (forward vs. forward+backward), since the backward pass recomputes attention tiles. Head dimension is fixed at 128 (with 64 reported as a secondary point) and batch size is held constant, so that N is isolated as the primary axis.

Platform axes. Two further axes are coupled to hardware rather than swept freely. *GPU generation*: FA1/FA2 and the traditional baselines run on Ampere (A100), FA3 on Hopper (H100), and FA4 on Blackwell (RTX PRO 6000 Blackwell). *Precision*: BF16 is used on all platforms for comparability, with FP8 added on Hopper (FA3) and FP4/MXFP8 on Blackwell (FA4), since these low-precision modes exist only on the newer hardware. We therefore report BF16 as the common axis and treat FP8/FP4 as platform-specific extensions. These are single-GPU spot-rented instances; the full sweep requires only a few GPU-hours per platform.

Metrics and protocol. For each configuration we record forward and forward+backward latency, throughput (tokens/s and TFLOP/s), peak GPU memory, maximum N before out-of-memory, and, for approximate methods and low-precision kernels, numerical error relative to exact attention. All compared methods are implemented in the same language (Python/PyTorch) and run on the same machine per platform, whose hardware and software we report. Each configuration is warmed up and repeated at least ten times per input; we report the *median* running time and its spread (minimum–maximum, or interquartile range), and we plot running time against sequence length for each method. Correctness is verified by asserting that each exact method agrees with naive attention within floating-point tolerance.

Reproducibility. We pin and report GPU model, driver, CUDA, PyTorch, and kernel versions, and release all benchmark scripts. The main practical risks, the alpha-stage FlashAttention-4 kernel and precision-dependent numerical behavior, are documented as part of the limitations analysis (RQ6 and RQ7).

5 Team Organization

5.1 Role Assignments

Table 1 assigns each member a primary area of responsibility that maps onto the structure of the study. All members contribute to the final paper and presentation.

Table 1: Primary role assignments.

Member	Primary responsibility
Shaaz E Jahan	Mathematical and algorithmic preliminaries for all methods: the naive implementation, each FlashAttention variant, and the baselines.
Jason Fakidis	The primary study: experiments comparing FlashAttention-1 against the traditional baselines.
Bahaa Al-Juboori	The lineage study: FA1/2/3/4 comparisons across GPU generations (Ampere, Hopper, Blackwell).
Aneet Kaur	Validation and discussion: verifying each experiment meets the outcomes set by the research questions, interpreting the results, and preparing the discussion.

5.2 Timeline

Table 2 gives the week-by-week plan for Weeks 8 to 14. Work is sequenced so that the preliminaries precede implementation, the two experimental studies run in parallel once the environment is ready, and validation follows the collection of results.

Table 2: Project timeline (Weeks 8 to 14).

Week	Activity
8	Finalize scope, methods, and research questions; stand up the shared repository; secure GPU access. Shaaz begins the mathematical and algorithmic preliminaries.
9	Preliminaries completed; implement and verify the baselines and the benchmark harness (timing, memory, out-of-memory capture).
10	Pilot runs: confirm FA1/FA2 and the baselines run correctly on Ampere, and confirm FA3 (Hopper) and FA4 (Blackwell) compatibility.
11	Jason runs the primary study (FA1 vs. baselines); Bahaa begins the lineage study (FA1/2/3/4 across Ampere, Hopper, Blackwell).
12	Complete both studies; collect at least five repeated measurements per input and record the median and spread.
13	Aneet validates results against the research-question outcomes; interpret findings and draft the discussion; produce figures and tables. Prepare and rehearse the presentation (delivered this week); submit slides as a PDF.
14	Finalize the paper and repository documentation; submit the contribution statement and peer evaluation.

References

[1] Alok Aggarwal and Jeffrey S. Vitter. 1988. The Input/Output Complexity of Sorting and Related Problems. *Commun. ACM* 31, 9, 1116–1127.

[2] Jimmy Ba, Geoffrey Hinton, Volodymyr Mnih, Joel Z. Leibo, and Catalin Ionescu. 2016. Using Fast Weights to Attend to the Recent Past. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:1610.06258.

[3] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. 2015. Neural Machine Translation by Jointly Learning to Align and Translate. In *International Conference on Learning Representations (ICLR)*. arXiv:1409.0473.

[4] Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamás Sarlós, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy Colwell, and Adrian Weller. 2021. Rethinking Attention with Performers. In *International Conference on Learning Representations (ICLR)*. arXiv:2009.14794.

[5] Tri Dao. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *International Conference on Learning Representations (ICLR)*. arXiv:2307.08691.

[6] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2205.14135.

[7] Sara Hooker. 2020. The Hardware Lottery. arXiv preprint. arXiv:2009.06489.

[8] Andrei Ivanov, Nikoli Dryden, Tal Ben-Nun, Shigang Li, and Torsten Hoefler. 2021. Data Movement Is All You Need: A Case Study on Optimizing Transformers. *Proceedings of Machine Learning and Systems (MLSys)* 3 (2021), 711–732.

[9] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. 2020. Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. In *International Conference on Machine Learning (ICML)*. 5156–5165.

[10] Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. 2020. Reformer: The Efficient Transformer. In *International Conference on Learning Representations (ICLR)*. arXiv:2001.04451.

[11] Maxim Milakov and Natalia Gimelshein. 2018. Online normalizer calculation for softmax. arXiv preprint. arXiv:1805.02867.

[12] Markus N. Rabe and Charles Staats. 2021. Self-attention Does Not Need $O(n^2)$ Memory. arXiv preprint. arXiv:2112.05682.

[13] Jürgen Schmidhuber. 1992. Learning to Control Fast-Weight Memories: An Alternative to Dynamic Recurrent Networks. *Neural Computation* 4, 1 (1992), 131–139.

[14] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2024. FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-Precision. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2407.08608.

[15] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:1706.03762.

[16] Sinong Wang, Belinda Z. Li, Madian Khabsa, Han Fang, and Hao Ma. 2020. Linformer: Self-Attention with Linear Complexity. In *arXiv preprint*. arXiv:2006.04768.

[17] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Commun. ACM* 52, 4 (2009), 65–76.

[18] Ted Zadouri et al. 2026. FlashAttention-4: Algorithm and Kernel Pipelining Co-Design for Asymmetric Hardware Scaling. arXiv preprint. arXiv:2603.05451.

A Requirements Coverage and Rubric Mapping

This appendix lists every deliverable requirement and rubric item for the research proposal and maps each to where this document satisfies it. Table 3 covers the Option B required-content items; Table 4 covers the grading rubric; the remaining items cover format and submission requirements.

Table 3: Option B required-content items and where each is addressed.

Required content	Where addressed
Problem and motivation	Sec. 1 (Problem and Motivation)
Modern algorithm: which, cited, with year	Sec. 3.1, "Choice of modern algorithm" (FlashAttention, 2022, with the 2023/2024/2026 variants)
Modern algorithm: why an advancement	Sec. 3.1, "Why it is an advancement"
Modern algorithm: problem/application	Sec. 3.1, "Application"
Baseline selection (2–4 traditional)	Sec. 3.2 (naive, Rabe-Staats, Performer, Linformer; SDPA/cuDNN as reference)
Research questions: claimed advantages	Sec. 4.1, "Claimed advantages" (RQ1–RQ3)
Research questions: conditions	Sec. 4.1, "Conditions" (RQ4–RQ5)
Research questions: limitations	Sec. 4.1, "Potential limitations" (RQ6–RQ7)
Research questions: experiments	Sec. 4.2 (Experimental Plan)
Team plan: roles and timeline	Sec. 5 (Role Assignments and Timeline)

Table 4: Research Proposal rubric (15% of project grade) and how this document addresses each criterion.

Criterion	Wt.	How addressed
Problem Definition & Motivation	25%	Sec. 1: first-principles problem statement and two-paragraph motivation
Literature Review	35%	Sec. 2: five categories tracing the lineage, with 10+ cited sources and a theory table (in the supplementary appendices)
Plan & Methodology	30%	Sec. 4: research questions (RQ1–RQ7) and a detailed experimental plan with swept parameters, metrics, and reproducibility
Team Organization	10%	Sec. 5: role assignments and a week-by-week timeline

Format and submission requirements.

- *ACM template*: the document uses the ACM Primary Article (`acmart`, `sigconf`) template.
- *Length (1–2 pages)*: the main body targets the required length; appendices and references are supplementary.
- *Citations*: all sources are cited in ACM reference format via BibTeX.